
EMBARCADERO CONFERENCE
PATROCINADOR MASTER:
embarcadero
An Idera, Inc. Company

ESCALANDO APIS HORSE HORIZONTALMENTE COM DOCKER

VINICIUS SANCHEZ

Nesta palestra vamos além da teoria: compilar uma API Horse para Linux com o PAServer, colocar o NGINX na frente como load balancer e escalar tudo horizontalmente ao vivo com Docker Compose.

VINICIUS SANCHEZ

- Técnico em Informática e Informática para Internet
- Graduado em Sistemas de Informação
- MBA em Engenharia IA (cursando)
- Mais de 12 anos como programador
- Membro da Hashload
- Embarcadero MVP
- Certified Delphi Developer



MONTREAL

anuário INFORMÁTICA HOJE destaque do ano 2025

Great Place To Work. Certificada MAR 2026-MAR 2027 Brasil

BDMG

GOIÁS

Banpará

MRV

CAIXA

Santander

CEMIG

ANS Agência Nacional de Saúde Suplementar

BANCO DO BRASIL

Neoenergia

Tecnologia para um mundo mais seguro, eficiente e conectado.

Acesse todas as nossas redes!

MONTREAL

HORSE: O FRAMEWORK PARA APIS REST EM DELPHI

O QUE É O HORSE?

Horse é um framework

Minimalista para criar APIs REST em Delphi, inspirado no Express.js.

Permite subir uma API funcional com poucas linhas de Código.

EXEMPLO DE API COM HORSE

```
uses Horse;

begin
  THorse.Get('/ping',
    procedure(Req: THorseRequest; Res: THorseResponse)
    begin
      Res.Send('pong');
    end);

 THorse.Listen(9000);
end.
```

POR QUE HORSE?

- Sintaxe simples e direta
- Extensível por middlewares
- Open-source
- Com comunidade grande e ativa
- Versão 3.x com muitas novidades...

SIMPLICIDADE

Pouco código, mais produtividade.

FLEXIBILIDADE

Middleware para diversas necessidades.

CONFIANÇA

Open-source confiável e em constante evolução.

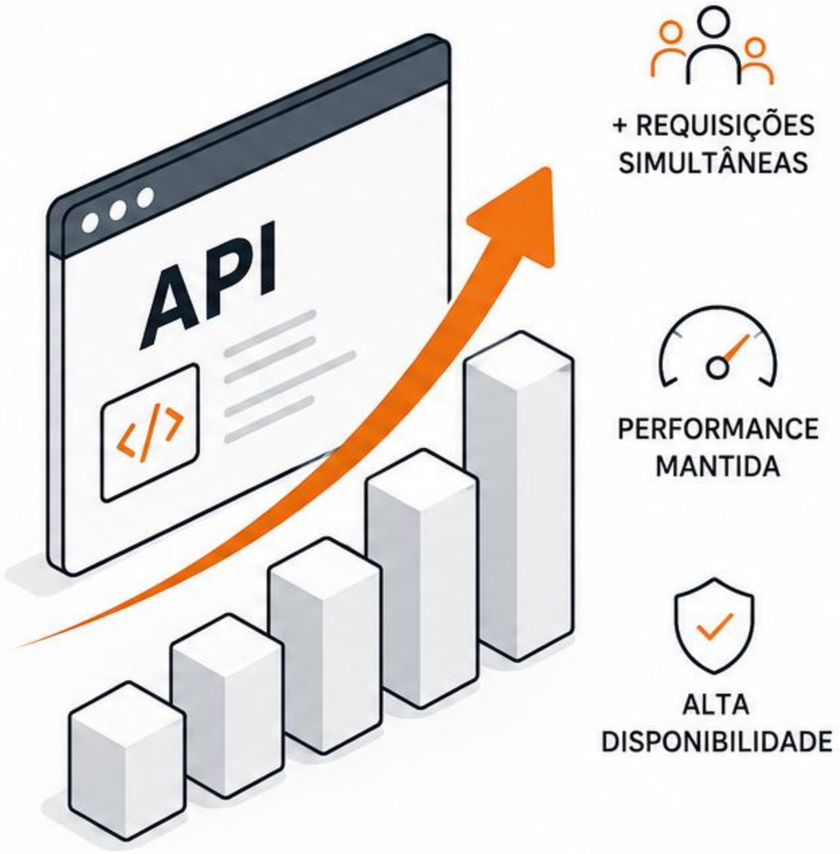
COMUNIDADE

Suporte ativo e colaboração da comunidade.



O QUE É ESCALAR UMA API?

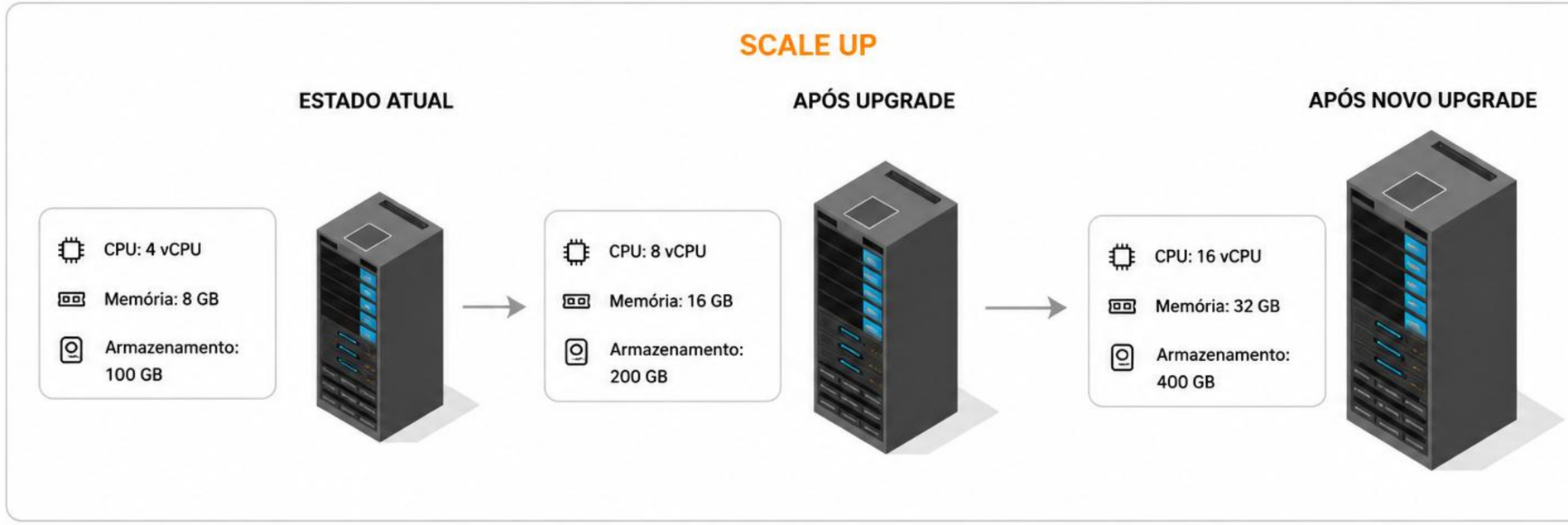
É aumentar a capacidade do sistema de atender mais requisições simultâneas, sem degradar performance nem disponibilidade.



ESCALAR VERTICALMENTE

Consiste basicamente em **aumentar os recursos** da máquina, como por exemplo, CPU e memória.

É simples de fazer, mas tem um **teto físico** e continua sendo um **único ponto de falha**.



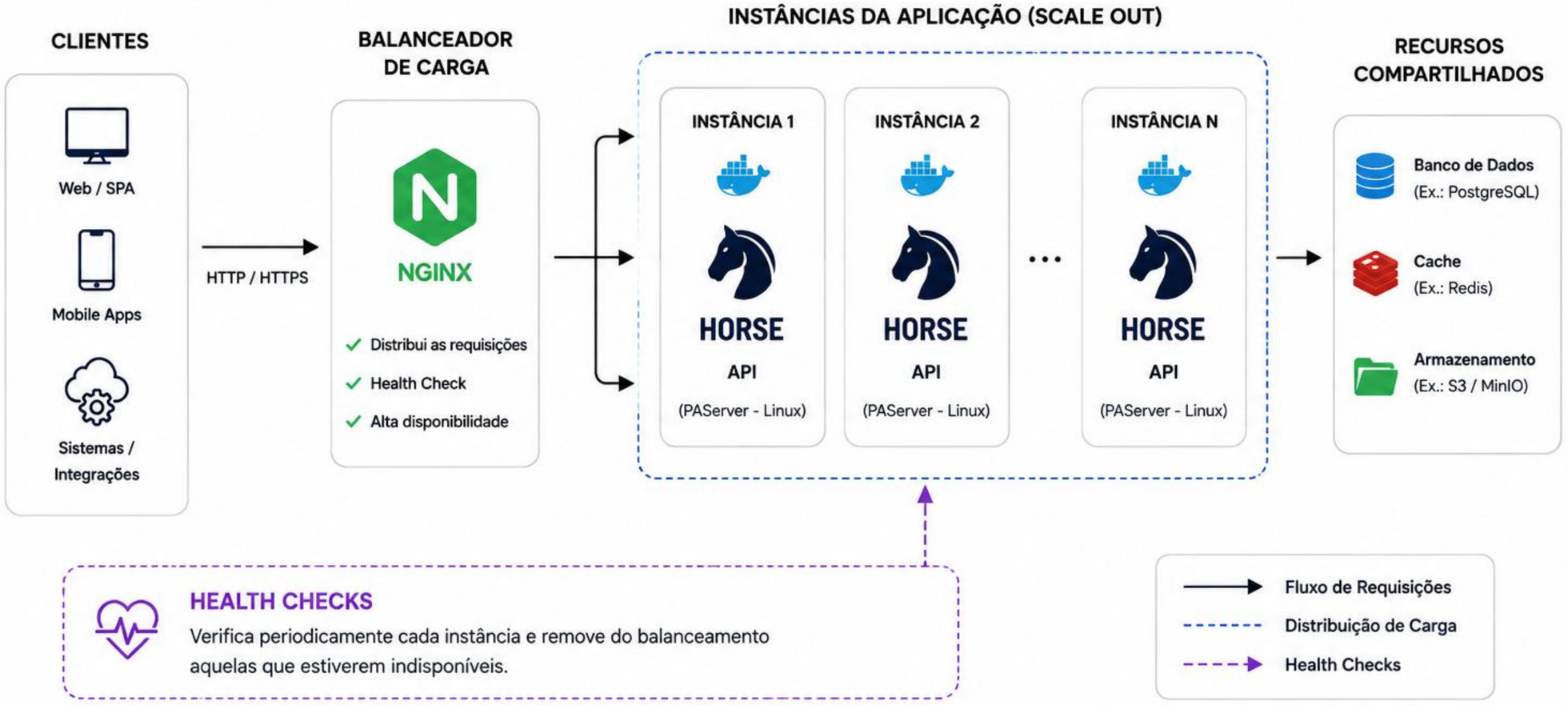
RESUMO: Aumenta-se os recursos de uma única máquina para suportar mais carga.

✓ Simples de implementar | ✗ Teto físico e único ponto de falha

ESCALAR HORIZONTALMENTE

Consiste em **aumentar** o número de instâncias rodando em paralelo e distribuir a carga entre elas.

Sem teto teórico, **tolera falhas**, mas exige um balanceador de carga na frente.



VERTICAL X HORIZONTAL

Vertical é mais simples de implementar, mas tem limite físico e é ponto único de falha.
Horizontal exige orquestração e um load balancer, mas escala sem teto teórico e tolera a queda de instâncias individuais.

SCALE-UP

Aumentar capacidade verticalmente



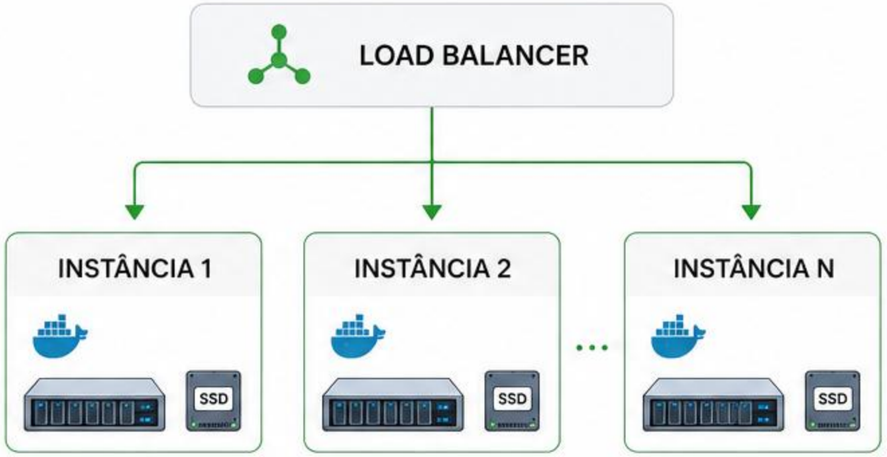
MAIS RECURSOS NO MESMO SERVIDOR



- Mais simples de implementar
- Limite físico de expansão
- Ponto único de falha

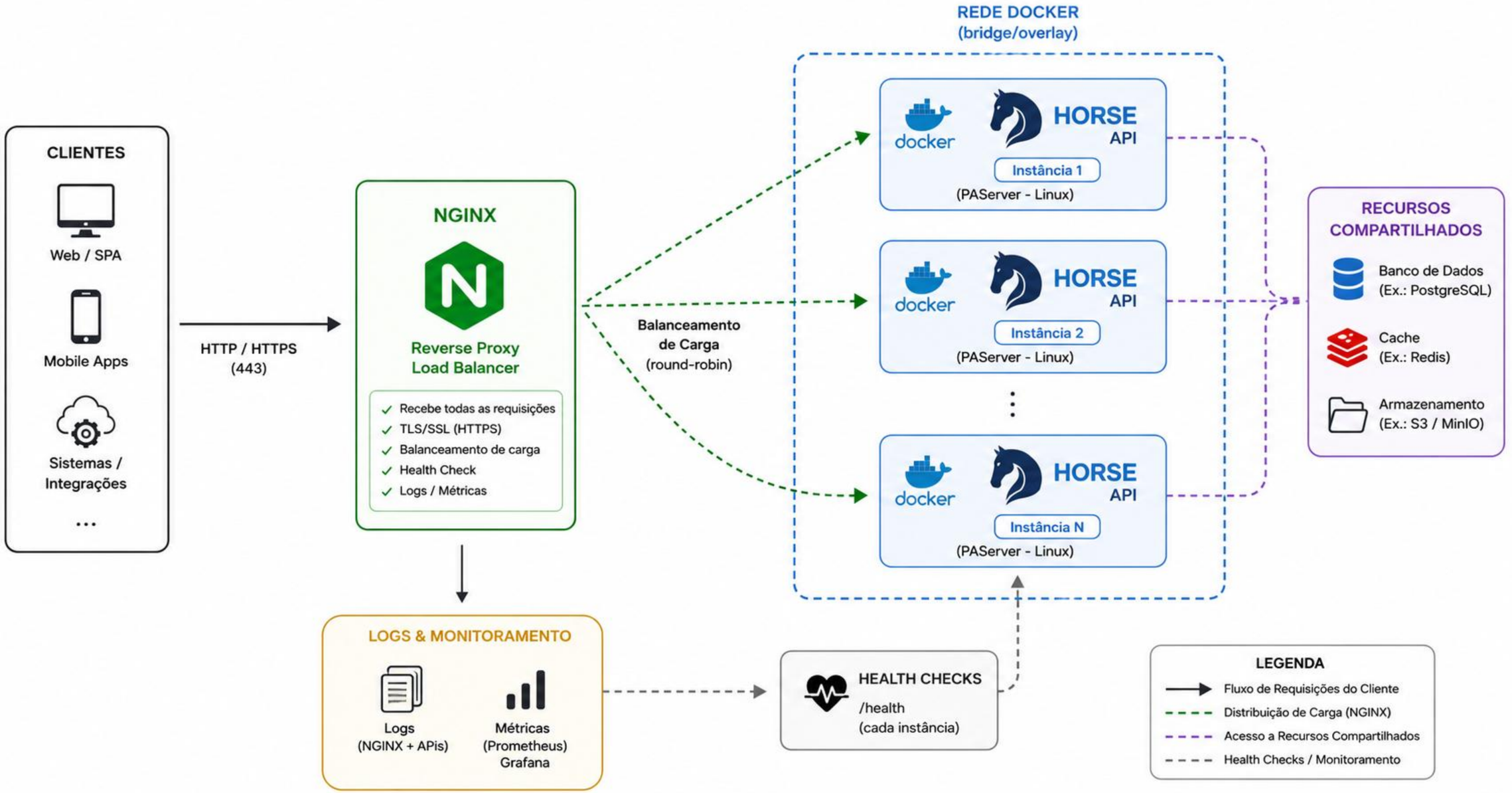
SCALE-OUT

Adicionar mais instâncias horizontalmente



- Escala sem teto teórico
- Tolerância a falha (alta disponibilidade)
- Crescimento sob demanda

RESUMO: Use Scale-Up quando precisar de simplicidade.
Use Scale-Out quando precisar de alta disponibilidade, resiliência e crescimento contínuo.


REDE DOCKER
(bridge/overlay)
CLIENTES
Web / SPA
Mobile Apps
Sistemas / Integrações
...
HTTP / HTTPS
(443)
NGINX
Reverse Proxy
Load Balancer
Recebe todas as requisições
TLS/SSL (HTTPS)
Balanceamento de carga
Health Check
Logs / Métricas
Balanceamento de Carga
(round-robin)
docker
HORSE
API
Instância 1
(PAServer - Linux)
Instância 2
(PAServer - Linux)
Instância N
(PAServer - Linux)
RECURSOS COMPARTILHADOS
Banco de Dados
(Ex.: PostgreSQL)
Cache
(Ex.: Redis)
Armazenamento
(Ex.: S3 / MinIO)
LOGS & MONITORAMENTO
Logs
(NGINX + APIs)
Métricas
(Prometheus)
Grafana
HEALTH CHECKS
/health
(cada instância)
LEGENDA
Fluxo de Requisições do Cliente
Distribuição de Carga (NGINX)
Acesso a Recursos Compartilhados
Health Checks / Monitoramento

```
program Samples;

{$APPTYPE CONSOLE}
{$R *.res}

uses Horse, System.SysUtils;

begin
  THorse.Get('/ping',
    procedure(Req: THorseRequest; Res: THorseResponse; Next: TProc)
    begin
      Res.Send(Format('Pong! Quem respondeu foi: %s', [GetEnvironmentVariable('HOSTNAME')]));
    end);

  THorse.Listen(9000,
    procedure
    begin
      Writeln(Format('Servidor rodando na porta %d... (host: %s)',
        [THorse.Port, GetEnvironmentVariable('HOSTNAME')]));
    end);
end.
```

DOCKER COMPOSE X KUBERNETES

Docker Compose escala muito bem numa máquina só. Com várias máquinas em produção, quem assume esse papel é um orquestrador como o Kubernetes.

DC DOCKER COMPOSE (demo)

```
docker compose up -d --scale api=3
```

- 1 máquina só, vários containers lado a lado
- Escala sob comando, manual, disparada por você
- NGINX faz o load balancing entre as réplicas
- Se a máquina cair, cai tudo junto
- Perfeito pra aprender o conceito e pra demos ao vivo

1 host | **Manual** | **Simples de começar**

K8s KUBERNETES (produção)

```
kubectl scale deployment api --replicas=3
```

- Cluster: várias máquinas (nodes) trabalhando juntas
- Escala automática por métrica (CPU, requisições) via HPA
- Load balancing nativamente
- Self-healing: pod morreu, o Kubernetes sobe outro sozinho
- Mesmo conceito da demo, só que distribuído e automatizado

Multi-host | **Auto-scaling** | **Self-healing**

i **RESUMO:** Docker Compose já mostra o conceito de escalar horizontalmente. Kubernetes é a mesma ideia levada pra produção multi-servidor, com automação por cima.

EMBARCADERO CONFERENCE 26

PATROCINADOR MASTER: embarcadero® An Idera, Inc. Company

QUER ME VER NA
#ECON27?
ACESSE O QRCODE E
AVALIE MINHA PALESTRA!

VINICIUS SANCHEZ

@viiniciussanchez

vinicius-sanchez

viniciuss.sanchez@gmail.com

(17) 9 9666-4049